# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: INTERNET PROGRAMMING**
**COURSE CODE: CSA4305**

**COURSE OUTCOME COVERED**

**CO-2 :** Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

**QUESTIONS:** **Total Marks: 30**

## Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.
(a) Analyze the possible reasons for lack of responsiveness in the existing layout.
(b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
(c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
(d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
(e) Evaluate the effectiveness of the responsive design and suggest further improvements.

## Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.
(a) Analyze the issues in the existing form validation mechanism.
(b) Design input fields for email and phone number with appropriate HTML attributes.
(c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
(d) Develop a mechanism to display **real-time error messages** for invalid inputs.
(e) Evaluate the validation system and suggest enhancements for better user experience and security.

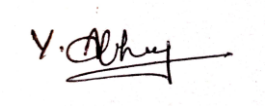
## Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.
(a) Analyze the CSS properties responsible for layout overlapping issues.
(b) Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
(c) Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
(d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
(e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

***Code of Conduct:***

*I Y. Abhay-192411219 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***



**RUBRICS FOR CALCULATION:**

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Concepts | 25 | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design | 30 | Well-planned, systematic implementation with optimal solution | Correct implementation with minor issues | Basic implementation with errors or inefficiencies | Incorrect or incomplete implementation |
| Use of Tools | 20 | Effective and advanced use of tools/technologies with proper configuration | Appropriate use of tools with correct execution | Limited or inconsistent use of tools | Incorrect or minimal use of tools |
| Analysis of Results | 15 | Insightful analysis with accurate interpretation and validation of results | Good analysis with reasonable interpretation | Basic analysis with limited insights | Poor or incorrect analysis of results |
| Documentation | 10 | Clear, well-structured documentation with diagrams, code, and results | Organized documentation with necessary details | Basic documentation with missing details | Poor or incomplete documentation |

# Answers:

# Experiment 1: Responsive Layout Design

## (a) Analysis

**Possible reasons for lack of responsiveness:**

- Fixed width elements (width: 1200px)
- No media queries
- Images without responsive sizing
- Improper use of margins and padding
- Using tables instead of Flexbox/Grid
- Missing viewport meta tag

## (b), (c), (d) Implementation

**index.html**

```
<!DOCTYPE html>
<html>
<head>
   <title>Responsive Layout</title>
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <link rel="stylesheet" href="style.css">
</head>

<body>

<header>
   <h1>Retail Store</h1>
</header>

<nav>
   <a href="#">Home</a>
   <a href="#">Products</a>
   <a href="#">Offers</a>
   <a href="#">Contact</a>
</nav>

<div class="container">

   <aside>
```

```
        <h3>Categories</h3>
        <p>Electronics</p>
        <p>Clothing</p>
        <p>Groceries</p>
    </aside>

    <main>
        <h2>Welcome</h2>
        <p>This is a responsive webpage using Flexbox.</p>
    </main>

</div>

<footer>
    ©2026 Retail Store
</footer>

</body>
</html>
```

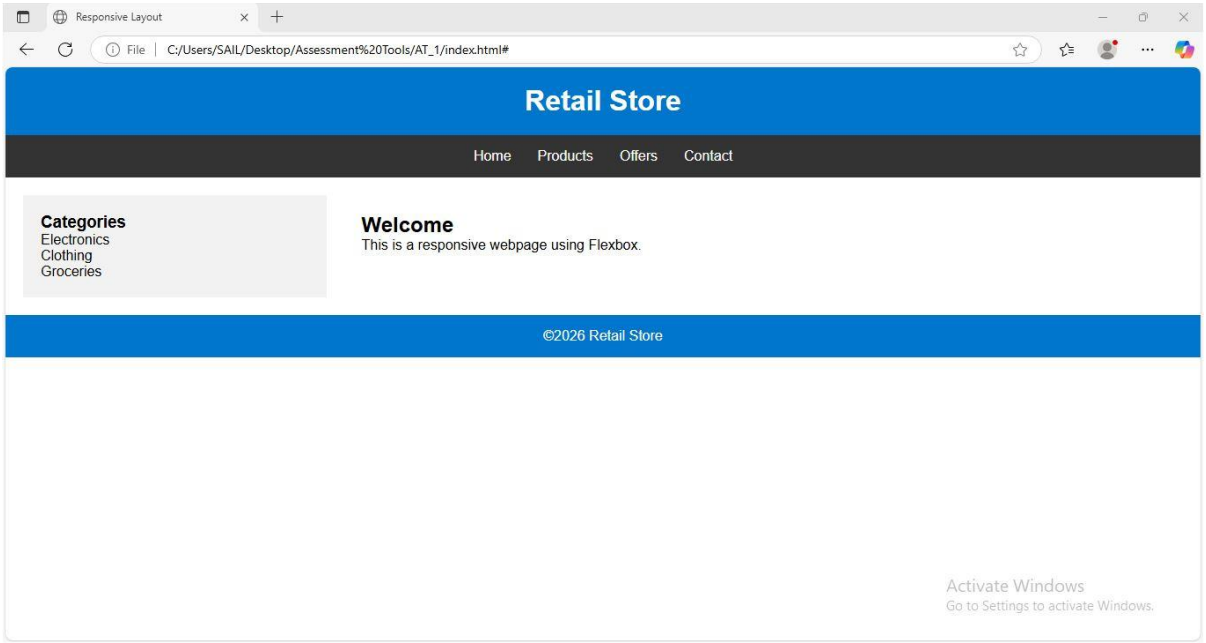
**style.css**

```
*{
margin:0;
padding:0;
box-sizing:border-box;
font-family:Arial;
}

header{
background:#0077cc;
color:white;
padding:20px;
text-align:center;
}

nav{
display:flex;
justify-content:center;
background:#333;
}

nav a{
color:white;
```

```
padding:15px;
text-decoration:none;
}

nav a:hover{
background:#555;
}

.container{
display:flex;
gap:20px;
padding:20px;
}

aside{
flex:1;
background:#f2f2f2;
padding:20px;
}

main{
flex:3;
background:white;
padding:20px;
}

footer{
background:#0077cc;
color:white;
text-align:center;
padding:15px;
}

@media(max-width:768px){

.container{
flex-direction:column;
}

nav{
flex-direction:column;
}

nav a{
text-align:center;
}
```

}

**Output:**




## (e) Evaluation

- Responsive on mobile, tablet and desktop.
- Proper spacing using Flexbox.
- No overlapping.
- Easy navigation.

### Further Improvements

- CSS Grid
- Bootstrap
- Better animations
- Dark mode
- Responsive images

# Experiment 2: Form Validation using JavaScript

## (a) Analysis

Problems:

- Invalid email accepted
- Invalid phone numbers accepted
- No error messages
- Data integrity issues

## (b), (c), (d) Implementation

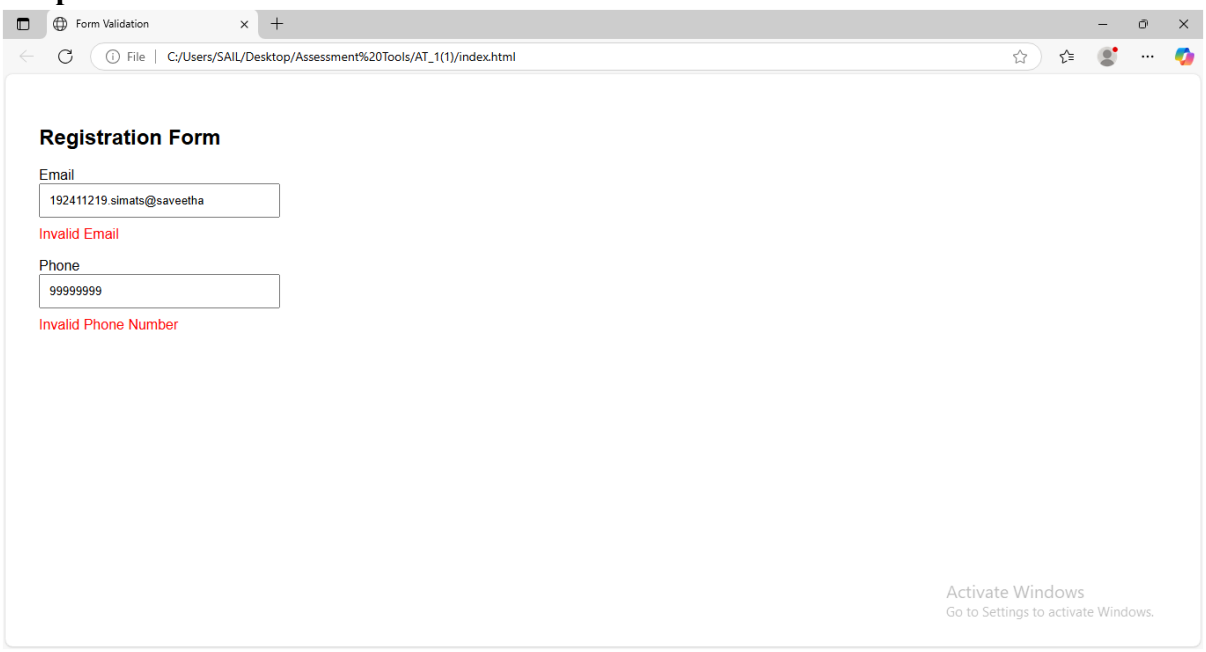
**index.html**

```
<!DOCTYPE html>
<html>

<head>

<title>Form Validation</title>

<style>

body{
font-family:Arial;
padding:30px;
}

input{
padding:10px;
width:250px;
margin-bottom:10px;
}

span{
color:red;
}
</style>

</head>

<body>
```

```
<h2>Registration Form</h2>

Email

<br>

<input type="email" id="email" onkeyup="validateEmail()">

<br>

<span id="emailError"></span>

<br><br>

Phone

<br>

<input type="text" id="phone" maxlength="10" onkeyup="validatePhone()">

<br>

<span id="phoneError"></span>

<script>

function validateEmail(){

let email=document.getElementById("email").value;

let pattern=/^[^ ]+@[^ ]+\.[a-z]{2,3}$/;

if(pattern.test(email))
document.getElementById("emailError").innerHTML="";

else
document.getElementById("emailError").innerHTML="Invalid Email";

}

function validatePhone(){

let phone=document.getElementById("phone").value;

let pattern=/^[0-9]{10}$/;

if(pattern.test(phone))
```

```
document.getElementById("phoneError").innerHTML="";

else
document.getElementById("phoneError").innerHTML="Invalid Phone Number";

}

</script>

</body>

</html>
```

**Output:**




## (e) Evaluation

Advantages

- Prevents invalid inputs.
- Immediate feedback.
- Better data quality.

Improvements

- Password validation
- OTP verification
- CAPTCHA
- Server-side validation

# Experiment 3: CSS Layout Debugging

## (a) Analysis

Causes of overlapping:

- Absolute positioning
- Fixed widths
- Large margins
- Missing responsive design
- Improper float usage

## (b) CSS Box Model

- Content
- Padding
- Border
- Margin

These determine element spacing and size.

## (c) & (d) Implementation

**index.html**

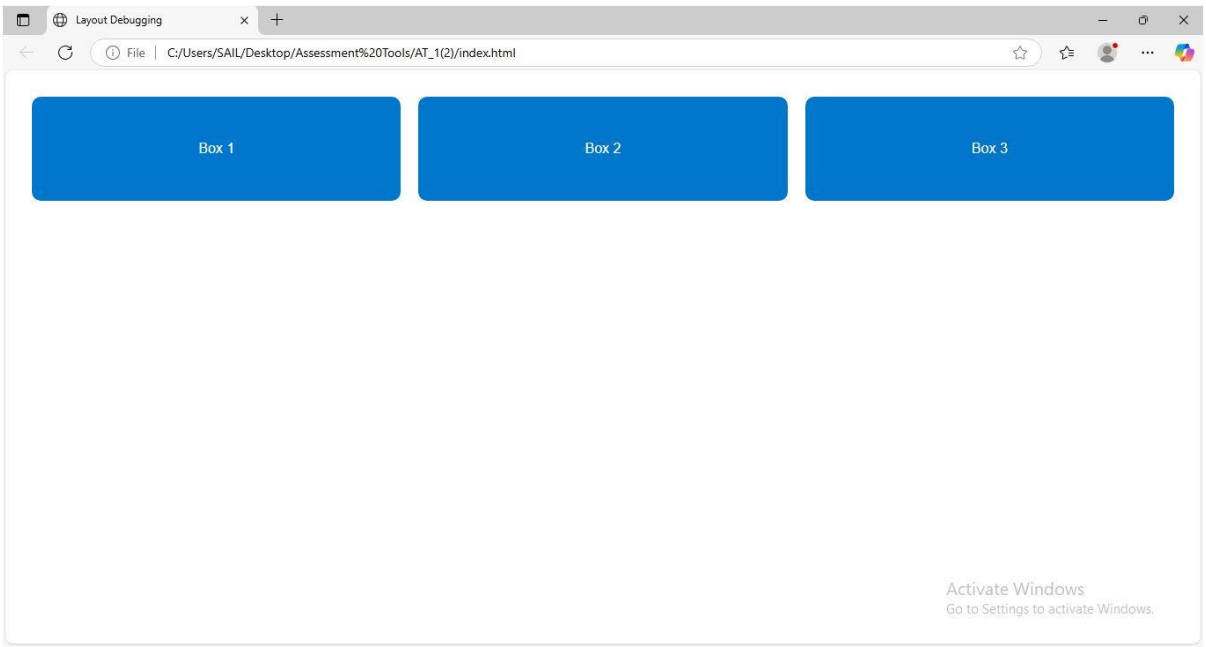
```
<!DOCTYPE html>
<html>

<head>

<title>Layout Debugging</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

<div class="box">Box 1</div>
```

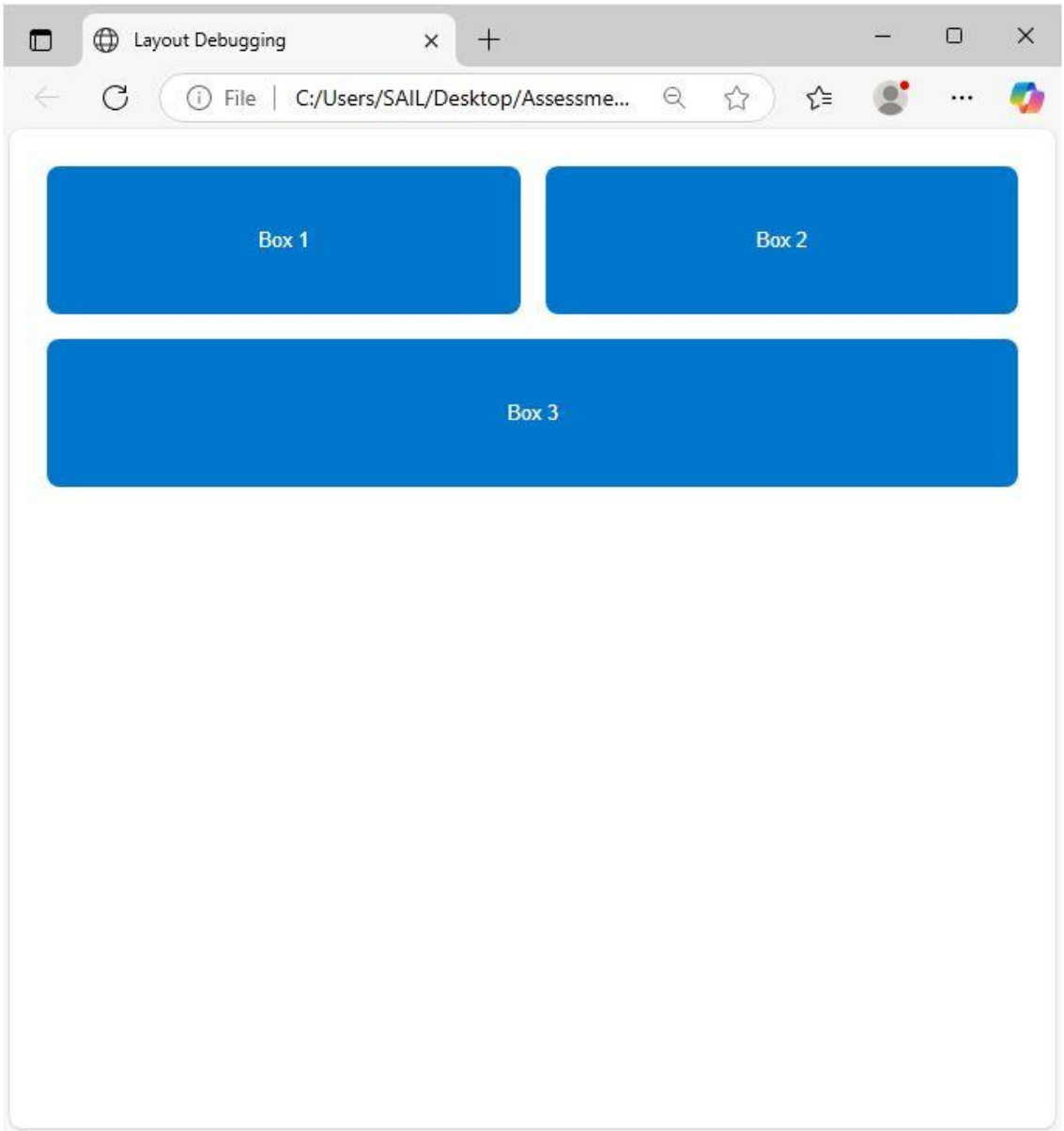
```
<div class="box">Box 2</div>

<div class="box">Box 3</div>

</div>

</body>

</html>
```

**style.css**

```
*{
margin:0;
padding:0;
box-sizing:border-box;
}

body{
padding:30px;
font-family:Arial;
}

.container{
display:flex;
gap:20px;
flex-wrap:wrap;
}

.box{
flex:1 1 250px;
background:#0077cc;
color:white;
padding:50px;
text-align:center;
border-radius:10px;
}

@media(max-width:768px){

.container{
flex-direction:column;
}

}
```

**Output:**


Layout Debugging
File | C:/Users/SAIL/Desktop/Assessment%20Tools/AT_1(2)/index.html
Box 1
Box 2
Box 3
Activate Windows
Go to Settings to activate Windows.


Layout Debugging
File | C:/Users/SAIL/Desktop/Assessme...
Box 1
Box 2
Box 3

## (e) Evaluation

Results:

- No overlapping
- Responsive layout
- Proper spacing
- Better readability

Best Practices:

- Use Flexbox/Grid instead of floats.
- Apply box-sizing: border-box.
- Use relative units (%, rem, vw).
- Test layouts on different screen sizes.
- Use media queries for responsiveness.